JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class Phaser

java.lang.Object
    java.util.concurrent.Phaser

```
public class Phaser
extends Object
```

A reusable synchronization barrier, similar in functionality to `CyclicBarrier` and `CountDownLatch` but supporting more flexible usage.

Registration. Unlike the case for other barriers, the number of parties *registered* to synchronize on a phaser may vary over time. Tasks may be registered at any time (using methods `register()`, `bulkRegister(int)`, or forms of constructors establishing initial numbers of parties), and optionally deregistered upon any arrival (using `arriveAndDeregister()`). As is the case with most basic synchronization constructs, registration and deregistration affect only internal counts; they do not establish any further internal bookkeeping, so tasks cannot query whether they are registered. (However, you can introduce such bookkeeping by subclassing this class.)

Synchronization. Like a `CyclicBarrier`, a `Phaser` may be repeatedly awaited. Method `arriveAndAwaitAdvance()` has effect analogous to `CyclicBarrier.await`. Each generation of a phaser has an associated phase number. The phase number starts at zero, and advances when all parties arrive at the phaser, wrapping around to zero after reaching `Integer.MAX_VALUE`. The use of phase numbers enables independent control of actions upon arrival at a phaser and upon awaiting others, via two kinds of methods that may be invoked by any registered party:

- **Arrival.** Methods `arrive()` and `arriveAndDeregister()` record arrival. These methods do not block, but return an associated *arrival phase number*; that is, the phase number of the phaser to which the arrival applied. When the final party for a given phase arrives, an optional action is performed and the phase advances. These actions are performed by the party triggering a

phase advance, and are arranged by overriding method `onAdvance(int, int)`, which also controls termination. Overriding this method is similar to, but more flexible than, providing a barrier action to a `CyclicBarrier`.
- **Waiting.** Method `awaitAdvance(int)` requires an argument indicating an arrival phase number, and returns when the phaser advances to (or is already at) a different phase. Unlike similar constructions using `CyclicBarrier`, method `awaitAdvance` continues to wait even if the waiting thread is interrupted. Interruptible and timeout versions are also available, but exceptions encountered while tasks wait interruptibly or with timeout do not change the state of the phaser. If necessary, you can perform any associated recovery within handlers of those exceptions, often after invoking `forceTermination`. Phasers may also be used by tasks executing in a `ForkJoinPool`, which will ensure sufficient parallelism to execute tasks when others are blocked waiting for a phase to advance.

Termination. A phaser may enter a *termination* state, that may be checked using method `isTerminated()`. Upon termination, all synchronization methods immediately return without waiting for advance, as indicated by a negative return value. Similarly, attempts to register upon termination have no effect. Termination is triggered when an invocation of `onAdvance` returns `true`. The default implementation returns `true` if a deregistration has caused the number of registered parties to become zero. As illustrated below, when phasers control actions with a fixed number of iterations, it is often convenient to override this method to cause termination when the current phase number reaches a threshold. Method `forceTermination()` is also available to abruptly release waiting threads and allow them to terminate.

Tiering. Phasers may be *tiered* (i.e., constructed in tree structures) to reduce contention. Phasers with large numbers of parties that would otherwise experience heavy synchronization contention costs may instead be set up so that groups of sub-phasers share a common parent. This may greatly increase throughput even though it incurs greater per-operation overhead.

In a tree of tiered phasers, registration and deregistration of child phasers with their parent are managed automatically. Whenever the number of registered parties of a child phaser becomes non-zero (as established in the `Phaser(Phaser,int)` constructor, `register()`, or `bulkRegister(int)`), the child phaser is registered with its parent. Whenever the number of registered parties becomes zero as the result of an invocation of `arriveAndDeregister()`, the child phaser is deregistered from its parent.

Monitoring. While synchronization methods may be invoked only by registered parties, the current state of a phaser may be monitored by any caller. At any given moment there are `getRegisteredParties()` parties in total, of which `getArrivedParties()` have arrived at the current phase (`getPhase()`). When the remaining (`getUnarrivedParties()`) parties arrive, the phase advances. The values returned by these methods may reflect transient states and so are not in general useful for synchronization control. Method `toString()` returns snapshots of these state queries in a form convenient for informal monitoring.

Sample usages:

A `Phaser` may be used instead of a `CountDownLatch` to control a one-shot action serving a variable number of parties. The typical idiom is for the method setting this up to first register, then start the actions, then deregister, as in:

```
 void runTasks(List<Runnable> tasks) {
   final Phaser phaser = new Phaser(1); // "1" to register self
   // create and start threads
   for (final Runnable task : tasks) {
     phaser.register();
     new Thread() {
       public void run() {
         phaser.arriveAndAwaitAdvance(); // await all creation
         task.run();
       }
     }.start();
   }

   // allow threads to start and deregister self
   phaser.arriveAndDeregister();
 }
```

One way to cause a set of threads to repeatedly perform actions for a given number of iterations is to override `onAdvance`:

```
 void startTasks(List<Runnable> tasks, final int iterations) {
   final Phaser phaser = new Phaser() {
     protected boolean onAdvance(int phase, int registeredParties) {
       return phase >= iterations || registeredParties == 0;
     }
   };
   phaser.register();
   for (final Runnable task : tasks) {
     phaser.register();
     new Thread() {
       public void run() {
         do {
           task.run();
           phaser.arriveAndAwaitAdvance();
         } while (!phaser.isTerminated());
       }
     }.start();
   }
   phaser.arriveAndDeregister(); // deregister self, don't wait
 }
```

If the main task must later await termination, it may re-register and then execute a similar loop:

```
  // ...
  phaser.register();
  while (!phaser.isTerminated())
    phaser.arriveAndAwaitAdvance();
```

Related constructions may be used to await particular phase numbers in contexts where you are sure that the phase will never wrap around `Integer.MAX_VALUE`. For example:

```
void awaitPhase(Phaser phaser, int phase) {
  int p = phaser.register(); // assumes caller not already registered
  while (p < phase) {
    if (phaser.isTerminated())
      // ... deal with unexpected termination
    else
      p = phaser.arriveAndAwaitAdvance();
  }
  phaser.arriveAndDeregister();
}
```

To create a set of `n` tasks using a tree of phasers, you could use code of the following form, assuming a Task class with a constructor accepting a `Phaser` that it registers with upon construction. After invocation of `build(new Task[n], 0, n, new Phaser())`, these tasks could then be started, for example by submitting to a pool:

```
void build(Task[] tasks, int lo, int hi, Phaser ph) {
  if (hi - lo > TASKS_PER_PHASER) {
    for (int i = lo; i < hi; i += TASKS_PER_PHASER) {
      int j = Math.min(i + TASKS_PER_PHASER, hi);
      build(tasks, i, j, new Phaser(ph));
    }
  } else {
    for (int i = lo; i < hi; ++i)
      tasks[i] = new Task(ph);
      // assumes new Task(ph) performs ph.register()
  }
}
```

The best value of `TASKS_PER_PHASER` depends mainly on expected synchronization rates. A value as low as four may be appropriate for extremely small per-phase task bodies (thus high rates), or up to hundreds for extremely large ones.

Implementation notes: This implementation restricts the maximum number of parties to 65535. Attempts to register additional parties result in `IllegalStateException`. However, you can and should create tiered phasers to accommodate arbitrarily large sets of participants.

Since:

1.7

Constructor Summary

Constructors

Constructor	Description
`Phaser()`	Creates a new phaser with no initially registered parties, no parent, and initial phase number 0.
`Phaser(int parties)`	Creates a new phaser with the given number of registered unarrived parties, no parent, and initial phase number 0.
`Phaser(Phaser parent)`	Equivalent to `Phaser(parent, 0)`.
`Phaser(Phaser parent, int parties)`	Creates a new phaser with the given parent and number of registered unarrived parties.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`int`	`arrive()`	Arrives at this phaser, without waiting for others to arrive.
`int`	`arriveAndAwaitAdvance()`	Arrives at this phaser and awaits others.
`int`	`arriveAndDeregister()`	Arrives at this phaser and deregisters from it without waiting for others to arrive.
`int`	`awaitAdvance(int phase)`	Awaits the phase of this phaser to advance from the given phase value, returning

		immediately if the current phase is not equal to the given phase value or this phaser is terminated.
int	**awaitAdvanceInterruptibly**(int phase)	Awaits the phase of this phaser to advance from the given phase value, throwing `InterruptedException` if interrupted while waiting, or returning immediately if the current phase is not equal to the given phase value or this phaser is terminated.
int	**awaitAdvanceInterruptibly**(int phase, long timeout, **TimeUnit** unit)	Awaits the phase of this phaser to advance from the given phase value or the given timeout to elapse, throwing `InterruptedException` if interrupted while waiting, or returning immediately if the current phase is not equal to the given phase value or this phaser is terminated.
int	**bulkRegister**(int parties)	Adds the given number of new unarrived parties to this phaser.
void	**forceTermination**()	Forces this phaser to enter termination state.
int	**getArrivedParties**()	Returns the number of registered parties that have arrived at the current phase of this phaser.
Phaser	**getParent**()	

		Returns the parent of this phaser, or `null` if none.
int	**getPhase**()	Returns the current phase number.
int	**getRegisteredParties**()	Returns the number of parties registered at this phaser.
Phaser	**getRoot**()	Returns the root ancestor of this phaser, which is the same as this phaser if it has no parent.
int	**getUnarrivedParties**()	Returns the number of registered parties that have not yet arrived at the current phase of this phaser.
boolean	**isTerminated**()	Returns `true` if this phaser has been terminated.
protected boolean	**onAdvance**(int phase, int registeredParties)	Overridable method to perform an action upon impending phase advance, and to control termination.
int	**register**()	Adds a new unarrived party to this phaser.
String	**toString**()	Returns a string identifying this phaser, as well as its state.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

Phaser

```
public Phaser()
```

Creates a new phaser with no initially registered parties, no parent, and initial phase number 0. Any thread using this phaser will need to first register for it.

Phaser

```
public Phaser(int parties)
```

Creates a new phaser with the given number of registered unarrived parties, no parent, and initial phase number 0.

Parameters:

`parties` - the number of parties required to advance to the next phase

Throws:

`IllegalArgumentException` - if parties less than zero or greater than the maximum number of parties supported

Phaser

```
public Phaser(Phaser parent)
```

Equivalent to `Phaser(parent, 0)`.

Parameters:

`parent` - the parent phaser

Phaser

```
public Phaser(Phaser parent,
              int parties)
```

Creates a new phaser with the given parent and number of registered unarrived parties. When the given parent is non-null and the given number of parties is greater than zero, this child phaser is registered with its parent.

Parameters:

`parent` - the parent phaser

parties - the number of parties required to advance to the next phase

Throws:

IllegalArgumentException - if parties less than zero or greater than the maximum number of parties supported

Method Detail

register

```
public int register()
```

Adds a new unarrived party to this phaser. If an ongoing invocation of onAdvance(int, int) is in progress, this method may await its completion before returning. If this phaser has a parent, and this phaser previously had no registered parties, this child phaser is also registered with its parent. If this phaser is terminated, the attempt to register has no effect, and a negative value is returned.

Returns:

the arrival phase number to which this registration applied. If this value is negative, then this phaser has terminated, in which case registration has no effect.

Throws:

IllegalStateException - if attempting to register more than the maximum supported number of parties

bulkRegister

```
public int bulkRegister(int parties)
```

Adds the given number of new unarrived parties to this phaser. If an ongoing invocation of onAdvance(int, int) is in progress, this method may await its completion before returning. If this phaser has a parent, and the given number of parties is greater than zero, and this phaser previously had no registered parties, this child phaser is also registered with its parent. If this phaser is terminated, the attempt to register has no effect, and a negative value is returned.

Parameters:

parties - the number of additional parties required to advance to the next phase

Returns:

the arrival phase number to which this registration applied. If this value is negative, then this phaser has terminated, in which case registration has no effect.

Throws:

`IllegalStateException` - if attempting to register more than the maximum supported number of parties

`IllegalArgumentException` - if parties < 0

arrive

```
public int arrive()
```

Arrives at this phaser, without waiting for others to arrive.

It is a usage error for an unregistered party to invoke this method. However, this error may result in an `IllegalStateException` only upon some subsequent operation on this phaser, if ever.

Returns:

the arrival phase number, or a negative value if terminated

Throws:

`IllegalStateException` - if not terminated and the number of unarrived parties would become negative

arriveAndDeregister

```
public int arriveAndDeregister()
```

Arrives at this phaser and deregisters from it without waiting for others to arrive. Deregistration reduces the number of parties required to advance in future phases. If this phaser has a parent, and deregistration causes this phaser to have zero parties, this phaser is also deregistered from its parent.

It is a usage error for an unregistered party to invoke this method. However, this error may result in an `IllegalStateException` only upon some subsequent operation on this phaser, if ever.

Returns:

the arrival phase number, or a negative value if terminated

Throws:

`IllegalStateException` - if not terminated and the number of registered or unarrived parties would become negative

arriveAndAwaitAdvance

```
public int arriveAndAwaitAdvance()
```

Arrives at this phaser and awaits others. Equivalent in effect to `awaitAdvance(arrive())`. If you need to await with interruption or timeout, you can arrange this with an analogous construction using one of the other forms of the `awaitAdvance` method. If instead you need to deregister upon arrival, use `awaitAdvance(arriveAndDeregister())`.

It is a usage error for an unregistered party to invoke this method. However, this error may result in an `IllegalStateException` only upon some subsequent operation on this phaser, if ever.

Returns:

the arrival phase number, or the (negative) current phase if terminated

Throws:

IllegalStateException - if not terminated and the number of unarrived parties would become negative

awaitAdvance

```
public int awaitAdvance(int phase)
```

Awaits the phase of this phaser to advance from the given phase value, returning immediately if the current phase is not equal to the given phase value or this phaser is terminated.

Parameters:

phase - an arrival phase number, or negative value if terminated; this
argument is normally the value returned by a previous call to arrive or arriveAndDeregister.

Returns:

the next arrival phase number, or the argument if it is negative, or the (negative) current phase if terminated

awaitAdvanceInterruptibly

```
public int awaitAdvanceInterruptibly(int phase)
                              throws InterruptedException
```

Awaits the phase of this phaser to advance from the given phase value, throwing `InterruptedException` if interrupted while waiting, or returning

immediately if the current phase is not equal to the given phase value or this phaser is terminated.

Parameters:

phase - an arrival phase number, or negative value if terminated; this
argument is normally the value returned by a previous call to arrive or arriveAndDeregister.

Returns:

the next arrival phase number, or the argument if it is negative, or the (negative) current phase if terminated

Throws:

InterruptedException - if thread interrupted while waiting

awaitAdvanceInterruptibly

```
public int awaitAdvanceInterruptibly(int phase,
                                     long timeout,
                                     TimeUnit unit)
                              throws InterruptedException,
                                     TimeoutException
```

Awaits the phase of this phaser to advance from the given phase value or the given timeout to elapse, throwing InterruptedException if interrupted while waiting, or returning immediately if the current phase is not equal to the given phase value or this phaser is terminated.

Parameters:

phase - an arrival phase number, or negative value if terminated; this
argument is normally the value returned by a previous call to arrive or arriveAndDeregister.

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

the next arrival phase number, or the argument if it is negative, or the (negative) current phase if terminated

Throws:

InterruptedException - if thread interrupted while waiting

TimeoutException - if timed out while waiting

forceTermination

```
public void forceTermination()
```

Forces this phaser to enter termination state. Counts of registered parties are unaffected. If this phaser is a member of a tiered set of phasers, then all of the phasers in the set are terminated. If this phaser is already terminated, this method has no effect. This method may be useful for coordinating recovery after one or more tasks encounter unexpected exceptions.

getPhase

```
public final int getPhase()
```

Returns the current phase number. The maximum phase number is `Integer.MAX_VALUE`, after which it restarts at zero. Upon termination, the phase number is negative, in which case the prevailing phase prior to termination may be obtained via `getPhase() + Integer.MIN_VALUE`.

Returns:

the phase number, or a negative value if terminated

getRegisteredParties

```
public int getRegisteredParties()
```

Returns the number of parties registered at this phaser.

Returns:

the number of parties

getArrivedParties

```
public int getArrivedParties()
```

Returns the number of registered parties that have arrived at the current phase of this phaser. If this phaser has terminated, the returned value is meaningless and arbitrary.

Returns:

the number of arrived parties

getUnarrivedParties

```
public int getUnarrivedParties()
```

Returns the number of registered parties that have not yet arrived at the current phase of this phaser. If this phaser has terminated, the returned value is meaningless and arbitrary.

Returns:

the number of unarrived parties

getParent

```
public Phaser getParent()
```

Returns the parent of this phaser, or `null` if none.

Returns:

the parent of this phaser, or null if none

getRoot

```
public Phaser getRoot()
```

Returns the root ancestor of this phaser, which is the same as this phaser if it has no parent.

Returns:

the root ancestor of this phaser

isTerminated

```
public boolean isTerminated()
```

Returns `true` if this phaser has been terminated.

Returns:

true if this phaser has been terminated

onAdvance

```
protected boolean onAdvance(int phase,
                            int registeredParties)
```

Overridable method to perform an action upon impending phase advance, and to control termination. This method is invoked upon arrival of the party advancing this phaser (when all other waiting parties are dormant). If this method returns `true`, this phaser will be set to a final termination state upon advance, and subsequent calls to `isTerminated()` will return true. Any (unchecked) Exception or Error thrown by an invocation of this method is propagated to the party attempting to advance this phaser, in which case no advance occurs.

The arguments to this method provide the state of the phaser prevailing for the current transition. The effects of invoking arrival, registration, and waiting methods on this phaser from within `onAdvance` are unspecified and should not be relied on.

If this phaser is a member of a tiered set of phasers, then `onAdvance` is invoked only for its root phaser on each advance.

To support the most common use cases, the default implementation of this method returns `true` when the number of registered parties has become zero as the result of a party invoking `arriveAndDeregister`. You can disable this behavior, thus enabling continuation upon future registrations, by overriding this method to always return `false`:

```
Phaser phaser = new Phaser() {
  protected boolean onAdvance(int phase, int parties) { return false; }
}
```

Parameters:

`phase - the current phase number on entry to this method, before this phaser is advanced`

`registeredParties - the current number of registered parties`

Returns:

`true if this phaser should terminate`

toString

```
public String toString()
```

Returns a string identifying this phaser, as well as its state. The state, in brackets, includes the String "phase = " followed by the phase number, "parties = " followed by the number of registered parties, and "arrived = " followed by the number of arrived parties.

Overrides:

toString in class Object

Returns:

a string identifying this phaser, as well as its state

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.